

02 Embodied AI Applications in Smart Home Assistant

1. Brief Game Instructions

Note: The large language model used in this lesson is cloud-based. Before starting, please connect the robot to the internet using an Ethernet cable or switch it to local area network (LAN) mode.

Say the wake-up word (the wake-up word depends on the firmware flashed, and by default, the wake-up word for our factory firmware is "Hello, HiWonder") to activate WonderEcho Pro. It will respond with "I am here."

Once activated, you can control MentorPi using voice commands—for example, "Go to the zoo to see what animals are there, then come back and tell me. After the command is issued, the terminal will display the recognized speech, while the "AI Voice Interaction Box" will verbally respond with the AI-generated answer and execute the corresponding actions simultaneously.

2. Preparation

2.1 Verifying the WonderEcho Pro Firmware

By default, the wake-up word for the WonderEcho Pro factory firmware is "Hello, Hiwonder." If you wish to change the wake-up word to "MentorPi," please refer to the tutorial titled "**20.1 Voice Module Overview and Installation\ 03 Firmware Flashing Method**" for step-by-step instructions.

If you have previously flashed the device with different firmware—such as the version with the wake word “小幻小幻” or “mentorpi”—you must use the corresponding wake word to activate the device.

For example, if the “小\幻小\幻” firmware was installed, the correct wake word would be “小\幻小\幻”; if the “mentorpi” firmware was installed, the wake word would be “mentorpi.”

In the following tutorial, we will use the default factory wake word “Hello, Hiwonder” for demonstration purposes.

2.2 Obtaining and Configuring the Large Model API Key

By default, the program does not include the configuration of the Large AI Model-related API keys. Before activating features related to the Large AI Model, please refer to the section **"02 Multimodal Large Model Applications > 01 Obtaining and Configuring Large Model API Keys"** to configure the necessary keys. This step is mandatory and cannot be skipped, as it is crucial for the proper functioning and experience of the large model features.

2.3 Network Configuration

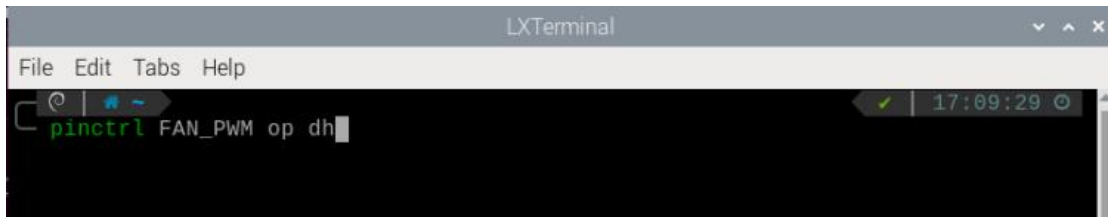
Note: The large language model used in this lesson is cloud-based. Before starting, please connect the robot to the internet using an Ethernet cable or switch it to local area network (LAN) mode.

The robot must be connected to the internet, either in STA (local network) mode or AP (direct connection) mode via Ethernet. For detailed instructions on network configuration, please refer to the course “20 Network Configuration Courses\ 01 Network Configuration Instructions”

2.4 Turn Off Cooling Fan

- 1) Click the icon in the top-left corner to open the terminal.
- 2) Type the command and press Enter to turn off the Raspberry Pi’s cooling fan, minimizing noise interference during recording.

pinctrl FAN_PWM op dh



2.5 Mapping

Refer to “12. Mapping Lesson -> Lesson 4 slam_toolbox Mapping Algorithm” to pre-build and save the map in advance.

3. Starting and Stopping the Game

1. The input commands must strictly observe case sensitivity and spacing.

2. The robot must be connected to the internet, either in STA (local network) mode or AP (direct connection) mode via Ethernet.

1) Start the robot and follow the instructions in “8. Remote Tool Installation Connection → 8.4 Docker Container Introduction and Entry” to connect via the VNC remote desktop software. Then, switch to the ROS2 environment and select the appropriate language version as needed.

2) Double-click  to launch the command bar. Enter the command and press Enter to disable the auto-start service.

3) Execute the following command to disable the app auto-start service.

~/stop_ros.sh

```
> ~/stop_ros.sh
```

4) Type the command and press Enter to launch the voice control mode. The

program will then start navigation.

ros2 launch large_models vllm_navigation.launch.py

```

/bin/zsh
/bin/zsh 80x24
-----
当前环境是ROS2|The current environment is ROS2
-----
LIDAR: LD19
CAMERA: ascamera
MACHINE: MentorPi_Mecanum
HOST: /
MASTER: /
ROS_DOMAIN_ID: 21
VERSION: |V2.0.0|2025-03-18|
[ros2 launch large_models vllm_navigation.launch.py]
  
```

5) When the command line shows the output below, initialization is complete.

You can then say the wake-up phrase, "Hello Hiwonder."

```

[agent_process-10] [INFO] [1742291623.374386995] [agent_process]: start
[tts_node-11] [INFO] [1742291624.176498371] [tts_node]: start
[vocal_detect-9] [INFO] [1742291624.184420787] [vocal_detect]: start
[ascamera_node-6] 2025-03-18 09:53:44[INFO] [CameraHp60c.cpp] [260] [startStreaming] start streaming
[agent_process-10] [INFO] [1742291624.996223930] [agent_process]: set model
  
```

6) When the command line shows the output below and the device announces "I am here," it means the voice device has been successfully activated. At this point, the system will begin recording the user's command.

```

[vocal_detect-7] [INFO] [1741419606.799872983] [vocal_detect]: asr...
[vocal_detect-7] [INFO] [1741419606.805699617] [vocal_detect]: Recording.....
[vocal_detect-7] [INFO] [1741419608.378862024] [vocal_detect]:
  
```

7) When the command line displays the output shown below, it indicates that WonderEcho Pro has printed the recognized speech.

```
[vocal_detect-14] [INFO] [1742556246.426869003] [vocal_detect]: 千?
[vocal_detect-14] [INFO] [1742556247.157503862] [vocal_detect]: 前往
[vocal_detect-14] [INFO] [1742556247.641239857] [vocal_detect]: 前往动物园
[vocal_detect-14] [INFO] [1742556248.126099371] [vocal_detect]: 前往动物园看看。
有
[vocal_detect-14] [INFO] [1742556248.597436945] [vocal_detect]: 前往动物园看看有
什么动
[vocal_detect-14] [INFO] [1742556249.085020183] [vocal_detect]: 前往动物园看看有
什么动物。然
[vocal_detect-14] [INFO] [1742556249.621643120] [vocal_detect]: 前往动物园看看有
什么动物，然后回来
[vocal_detect-14] [INFO] [1742556251.257122877] [vocal_detect]: 前往动物园看看有
什么动物，然后回来告诉我。
```

- 8) When the command line shows the output below, it indicates that the cloud-based large language model has successfully processed the user's command, thought through the instruction, and provided a verbal response ("response"), as well as designed an action that aligns with the user's command meaning.

The response is automatically generated by the large model, ensuring the accuracy of the meaning, though the wording and structure of the reply may vary.

```
[agent_process-15] [INFO] [1742556253.844343576] [agent_process]: publish llm re
sult:{"action":["move('动物园')", "vision('哪些动物')", "move('原点')", "play_au
dio()"], "response":"动物园探秘，速去速回"}
```

- 9) When the command line shows the output below, it indicates that one round of interaction has been completed. You can refer to Step 4) to reactivate the voice device using the wake-up word and begin a new round of interaction.

```
[object_transport-10] [INFO] [1741435647.610069156] [object_transport]: enable object transport
[vocal_detect-7] [INFO] [1741435647.623095741] [vocal_detect]: enable_wakeup
```

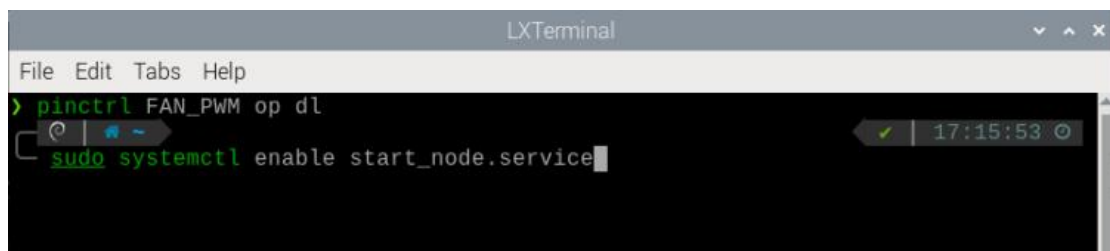
- 10) To exit this feature, simply press "Ctrl+C" in the terminal. If it doesn't close on the first attempt, you may need to press it multiple times.
- 11) Type the command and press Enter to turn on the Raspberry Pi cooling fan.

```
pinctrl FAN_PWM op dl
```



12) Enter the command and press the Enter key to start the game:

sudo systemctl enable start_node.service



4. How It Works

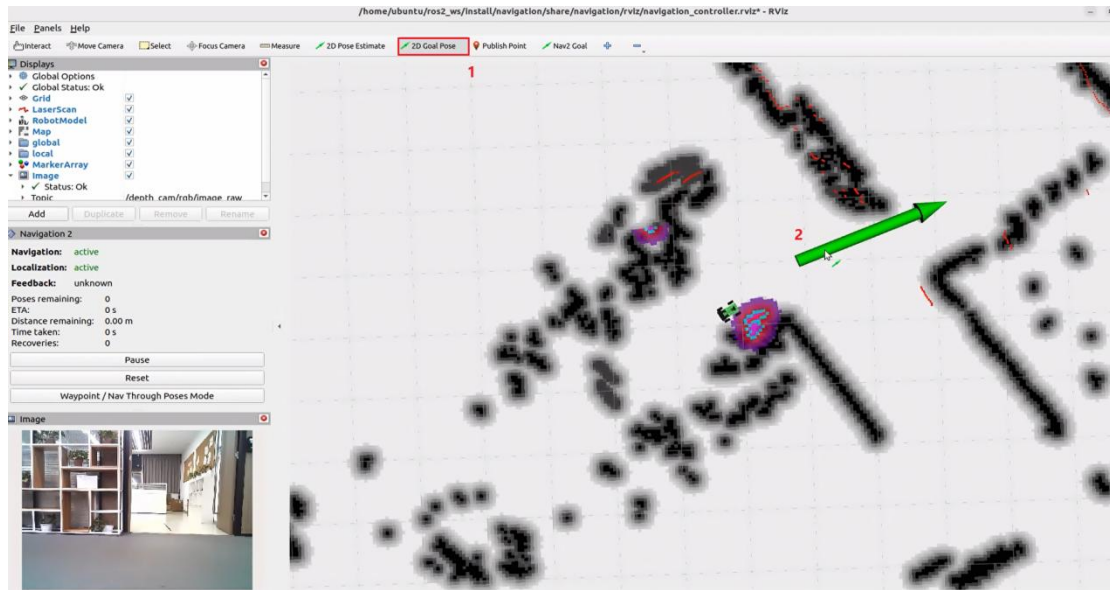
Once activated, you can freely give commands like: “Go to the zoo to see what animals are there, then come back and tell me.” The robot will navigate to the designated zoo location, count the animal models present, and return to the starting point to report its findings.

5. Modifying Navigation Positions

To update the navigation waypoints in the program, edit the following file:

ros2_ws/src/large_models/large_models/vllm_navigation.py

- 1) First, refer to “Section 4: Starting and Stopping the Game / Step 4” to launch the program and open the RViz map visualization. Use the 2D Goal Pose tool to set the desired navigation target directly on the map.



- 2) Then, switch back to the terminal to view the published target position parameters. Enter these parameter values into the program accordingly.

```
[navigation_controller-3] [INFO] [1742392582.005780583] [navigation_controller]: geometry_msgs.msg.PoseStamped(header=std_msgs.msg.Header(stamp=builtin_interfaces.msg.Time(sec=1742392581, nanosec=997458229), frame_id='map'), pose=geometry_msgs.msg.Pose(position=geometry_msgs.msg.Point(x=0.9955235123634338, y=-0.8725460171699524, z=0.0), orientation=geometry_msgs.msg.Quaternion(x=0.0, y=0.0, z=-0.5677173914973032, w=0.8232235197025761)))
```

```
23 position_dict = {"动物园": [1.2548, -0.0879, 0.0, 0.0, -0.1605], #xyzrpy, m,deg
24                  "超市": [1.1782, -1.1059, 0.0, 0.0, 0.0, -0.9938],
25                  "足球场": [2.49, -2.53, 0.0, 0.0, -1.76],
26                  "原点": [0.0, 0.0, 0.0, 0.0, 0.0]}
```

Each set of five numbers corresponds to:

x coordinate (meters)

y coordinate (meters)

roll angle (rotation around the x-axis, in degrees)

pitch angle (rotation around the y-axis, in degrees)

yaw angle (rotation around the z-axis, in degrees) Use the map's coordinate system as a reference to fill in the x and y values. Next, convert the quaternion value, for example:

Quaternion(x=0.0, y=0.0, z=-0.5677, w=0.8232)

to Euler angles (roll, pitch, yaw), which would be approximately:

roll $\approx 0^\circ$, pitch $\approx 0^\circ$, yaw $\approx -69.3^\circ$.

6. Brief Program Analysis

6.1 Analysis of launch File

The program is saved at:

/home/ubuntu/ros2_ws/src/large_models/launch/vllm_with_camera.launch.py

◆ Library Files Import

```
1 import os
2 from ament_index_python.packages import get_package_share_directory
3
4 from launch_ros.actions import Node
5 from launch_ros.actions import PushRosNamespace
6 from launch import LaunchDescription, LaunchService
7 from launch.substitutions import LaunchConfiguration
8 from launch.launch_description_sources import PythonLaunchDescriptionSource
9 from launch.actions import DeclareLaunchArgument, IncludeLaunchDescription, GroupAction, OpaqueFunction, TimerAction, ExecuteProcess
```

① os: Used for handling file paths and operating system-related functionalities.

② ament_index_python.packages.get_package_share_directory: Used to obtain the shared directory path of a ROS2 package.

③ launch_ros.actions.Node: Used to define a ROS2 node

④ launch.substitutions.LaunchConfiguration: Used to retrieve parameter configurations from a Launch file.

⑤ LaunchDescription, LaunchService: Used to define and launch a Launch file.

⑥ launch_description_sources PythonLaunchDescriptionSource: Used to include other Launch files.

⑦ launch.actions.IncludeLaunchDescription, DeclareLaunchArgument, OpaqueFunction: Used to define actions and parameters within a Launch file.

◆ Define launch_setup Function

```

11 def launch_setup(context):
12     jetauto_description_package_path = get_package_share_directory('mentorpi_description')
13     slam_package_path = get_package_share_directory('slam')
14     navigation_package_path = get_package_share_directory('navigation')
15     large_models_package_path = get_package_share_directory('large_models')
16
17     mode = LaunchConfiguration('mode', default=1)
18     mode_arg = DeclareLaunchArgument('mode', default_value=mode)
19     map_name = LaunchConfiguration('map', default='map_01').perform(context)
20     robot_name = LaunchConfiguration('robot_name', default=os.environ['HOST'])
21     master_name = LaunchConfiguration('master_name', default=os.environ['MASTER'])
22
23     map_name_arg = DeclareLaunchArgument('map', default_value=map_name)
24     master_name_arg = DeclareLaunchArgument('master_name', default_value=master_name)
25     robot_name_arg = DeclareLaunchArgument('robot_name', default_value=robot_name)
26
27     navigation_launch = IncludeLaunchDescription(
28         PythonLaunchDescriptionSource(os.path.join(navigation_package_path, 'launch/navigation.launch.py')),
29         launch_arguments={
30             'sim': 'false',
31             'map': map_name,
32             'robot_name': robot_name,
33             'master_name': master_name,
34             'use_teb': 'true',
35         }.items(),
36     )
37
38     navigation_controller_node = Node(
39         package='large_models',
40         executable='navigation_controller',
41         output='screen',
42         parameters=[{'map_frame': 'map', 'nav_goal': '/nav_goal'}]
43     )
44
45     rviz_node = ExecuteProcess(
46         cmd=['rviz2', 'rviz2', '-d', os.path.join(navigation_package_path, 'rviz/navigation_controller.rviz')],
47         output='screen'
48     )

```

① mode = LaunchConfiguration('mode', default=1): Defines a Launch parameter named mode, with a default value of 1.

② sdk_launch: Uses IncludeLaunchDescription to include the MentorPi_sdk.launch.py launch file from the large_models package, passing the mode parameter.

③ large_models_launch: Uses IncludeLaunchDescription to include the start.launch.py Launch file from the large_models package, passing the mode parameter.

◆ Define generate_launch_description function:

```

74 def generate_launch_description():
75     return LaunchDescription([
76         OpaqueFunction(function = launch_setup)
77     ])
78

```

◆ Main program entry:

```

79 if __name__ == '__main__':
80     # 创建一个LaunchDescription对象(create a LaunchDescription object)
81     ld = generate_launch_description()
82
83     ls = LaunchService()
84     ls.include_launch_description(ld)
85     ls.run()

```

① ld = generate_launch_description(): Generates a Launch description object.

② ls = LaunchService(): Creates a Launch service object.

③ ls.include_launch_description(ld): Adds the Launch description to the Launch service.

④ ls.run(): Starts the Launch service, executing all the defined Launch actions.

6.2 Analysis of Python Files

The program is saved at:

/home/ubuntu/ros2_ws/src/large_models/large_models/vllm_navigation.py

◆ Feature Package Import

```

5 import re
6 import time
7 import json
8 import rclpy
9 import threading
10 from speech import speech
11 from rclpy.node import Node
12 from sensor_msgs.msg import Image
13 from std_msgs.msg import String, Bool
14 from std_srvs.srv import Trigger, SetBool, Empty
15
16 from large_models.config import *
17 from large_models_msgs.srv import SetModel, SetContent, SetString, SetInt32
18
19 from interfaces.srv import SetPose2D
20 from rclpy.executors import MultiThreadedExecutor
21 from rclpy.callback_groups import ReentrantCallbackGroup

```

- ① **cv2**: Used for image processing and display with OpenCV.
- ② **json**: Used for handling JSON-formatted data.
- ③ **.sdk.fps as fps**: Used to calculate and display the frame rate (FPS).
- ④ **speech**: Modules related to large model voice interaction.
- ⑤ **large_models_msgs.srv**: Custom service type for large models.
- ⑥ **large_models.config**: Configuration file for large models.

◆ Modifying Navigation Positions

```
23 position_dict = {"动物园": [1.2548, -0.0879, 0.0, 0.0, -0.1605], #xyzrpy, m,deg
24                  "超市": [1.1782, -1.1059, 0.0, 0.0, 0.0, -0.9938],
25                  "足球场": [2.49, -2.53, 0.0, 0.0, 0.0, -1.76],
26                  "原点": [0.0, 0.0, 0.0, 0.0, 0.0]}
```

◆ LLM_PROMPT character string

```
75 LLM_PROMPT = """
76 **Role
77 You are a smart navigation vehicle equipped with a camera and speaker. You can move to different places, analyze visual input, and respond by playing audio. Based on u
78 r input, you need to generate the corresponding JSON command.
79
80 **Requirements
81 - For any user input, look up corresponding functions from the Action Function Library, and generate the proper JSON output.
82 - For each action sequence, include a concise (5-20 characters) and witty, varied response to make the interaction lively and engaging.
83 - Output only the JSON result, no analysis or extra text.
84 - Output format:
85 {
86   "action": ["xx", "xx"],
87   "response": "xx"
88 }
89
90 **Special Notes
91 The "action" field contains an ordered list of function names to be executed in sequence. If no matching function is found, return: "action": [].
92 The "response" field should contain a carefully crafted, short, humorous, and varied message (5-20 characters).
93
94 **Action Function Library
95 Move to a specified place: move('kitchen')
96 Return to starting point: move('origin')
97 Analyze current view: vision('What do you see?')
98 Play audio response: play_audio()
99
100 **Example
101 Input: Go to the front desk to see if the door is closed, and then come back and tell me
102 Output:
103 {
104   "action": ["move('front desk')", "vision('Is the door closed?')", "move('origin')", "play_audio()"],
105   "response": "On it, reporting soon!"
106 }
107 """
```

◆ VLLM_PROMPT character string

```
108 VLLM_PROMPT = """
109 As my robot butler, you should carefully observe the surrounding situation and give considerate and humane responses to the instructions given. Do not ask questions in r
110 return. The number of words should be between 20 and 40.
111 """
```

◆ VLLMObjectTransport Class

```

60 class VLLMNavigation(Node):
61     def __init__(self, name):
62         rclpy.init()
63         super().__init__(name)
64
65         self.action = []
66         self.response_text = ''
67         self.llm_result = ''
68         self.play_audio_finish = False
69         # self.llm_result = '\action\':[\move(\前台\)\', \vision(\大门有没有关\)\', \move(\原点\)\', \play_audio()\', \response\':\马上!\']
70         self.running = True
71         self.reach_goal = False
72         self.interrupt = False
73
74         timer_cb_group = ReentrantCallbackGroup()
75         self.tts_text_pub = self.create_publisher(String, 'tts_node/tts_text', 1)
76         # self.create_subscription(Image, 'depth_cam/rgb/image_raw', self.image_callback, 1)
77         self.create_subscription(String, 'agent_process/result', self.llm_result_callback, 1)
78         self.create_subscription(Bool, 'vocal_detect/wakeup', self.wakeup_callback, 1)
79         self.create_subscription(Bool, 'tts_node/play_finish', self.play_audio_finish_callback, 1, callback_group=timer_cb_group)
80         self.create_subscription(Bool, 'navigation_controller/reach_goal', self.reach_goal_callback, 1)
81         self.awake_client = self.create_client(SetBool, 'vocal_detect/enable_wakeup')
82         self.awake_client.wait_for_service()
83         self.set_mode_client = self.create_client(SetInt32, 'vocal_detect/set_mode')
84         self.set_mode_client.wait_for_service()
85         self.set_model_client = self.create_client(SetModel, 'agent_process/set_model')
86         self.set_model_client.wait_for_service()
87         self.set_prompt_client = self.create_client(SetString, 'agent_process/set_prompt')
88         self.set_prompt_client.wait_for_service()
89         self.set_vllm_content_client = self.create_client(SetContent, 'agent_process/set_vllm_content')
90         self.set_vllm_content_client.wait_for_service()
91         self.set_pose_client = self.create_client(SetPose2D, 'navigation_controller/set_pose')
92         self.set_pose_client.wait_for_service()
93
94         self.timer = self.create_timer(0.0, self.init_process, callback_group=timer_cb_group)

```

① self.llm_result: Stores the results returned by the LLM (Large Language Model).

② self.running: control the running status of the program

③ self.transport_finished: Marks whether the transportation task has been completed.

④ self.client: The voice API client, used for interacting with voice services.

⑤ self.tts_text_pub: ROS topic publisher for sending Text-to-Speech (TTS) text.

⑥ self.tts_text_pub: ROS topic publisher for sending Automatic Speech Recognition (ASR) results.

⑦ self.create_subscription: Subscribes to LLM results

◆ get_node_state Method

```

96     def get_node_state(self, request, response):
97         return response

```

return response: Returns the response object.

◆ init_process Method

```

99     def init_process(self):
100         self.timer.cancel()
101
102         msg = SetModel.Request()
103         msg.model = llm_model
104         msg.model_type = 'llm'
105         msg.api_key = api_key
106         msg.base_url = base_url
107         self.send_request(self.set_model_client, msg)
108
109         msg = SetString.Request()
110         msg.data = LLM_PROMPT
111         self.send_request(self.set_prompt_client, msg)
112
113         init_finish = self.create_client(Empty, 'navigation_controller/init_finish')
114         init_finish.wait_for_service()
115         speech.play_audio(start_audio_path)
116         threading.Thread(target=self.process, daemon=True).start()
117         self.create_service(Empty, '~/init_finish', self.get_node_state)
118         self.get_logger().info('\033[1;32m%s\033[0m' % 'start')

```

- ① SetModel.Request: Creates a request message to set the model.
- ② SetString.Request: Defines the request message for setting the prompt string.
- ③ set_pose_target: Sets the robot's initial pose.
- ④ Self.send_request: Sends a service request.
- ⑤ speech.play_audio: Plays an audio clip.
- ⑥ threading.Thread: Starts a new thread for image processing and tracking.

◆ Send_request Method

```

173     def send_request(self, client, msg):
174         future = client.call_async(msg)
175         while rclpy.ok():
176             if future.done() and future.result():
177                 return future.result()

```

- ① Client.call_async(msg): Asynchronously calls a service.
- ② future.done() and future.result(): Check if the service call has completed and retrieve the result.

◆ llm_result_callback method:

```

183     def llm_result_callback(self, msg):
184         self.llm_result = msg.data
185

```

Receives results from the agent_process/result topic and stores them in self.vllm_result.

◆ wakeup_callback method

```

179     def wakeup_callback(self, msg):
180         self.get_logger().info('wakeup interrupt')
181         self.interrupt = msg.data
182

```

Receives and processes wake-up signals from the vocal_detect/wakeup topic. This method is triggered when the user wakes up the system using voice commands.

◆ def move(self, position): method

```

133     def move(self, position):
134         self.get_logger().info('position: %s' % str(position))
135         msg = SetPose2D.Request()
136         p = position_dict[position]
137         msg.data.x = float(p[0])
138         msg.data.y = float(p[1])
139         msg.data.roll = p[2]
140         msg.data.pitch = p[3]
141         msg.data.yaw = p[4]
142         self.send_request(self.set_pose_client, msg)

```

① Parameter Reception: Receives a location name (e.g., "zoo", "supermarket", etc.)

② Position Lookup: Retrieves the corresponding coordinates and orientation data from the position_dict dictionary using p = position_dict[position].

③ Message Construction: Creates a SetPose2D.Request() message and sets the x and y coordinates, as well as the roll, pitch, and yaw angles.

④ Service Call: Sends the location request to the navigation controller

by calling `self.send_request(self.set_pose_client, msg)` through the `set_pose_client`.

This sequence enables the robot to navigate between predefined locations, forming the core functionality for spatial movement tasks.

◆ **play_audio(self) Method:**

```
144     def play_audio(self):
145         msg = String()
146         msg.data = self.response_text
147         self.get_logger().info(f'{self.response_text}')
148         while not self.play_audio_finish:
149             time.sleep(0.1)
150         self.play_audio_finish = False
151         self.tts_text_pub.publish(msg)
152         self.response_text = ''
```

Plays the text stored in `self.response_text` as audio. Sends a wake-up signal once audio playback is completed.

◆ **reach_goal_callback(self, msg) method**

① **Parameter Reception:** Receives a boolean message indicating whether the target has been reached.

② **Status Update:** Assigns the message data to the `self.reach_goal` variable.

This allows the system to receive feedback from the navigation controller and update the robot's navigation status, enabling it to know when the movement task is complete.

◆ **vision(self, query) Method:**

```
158     def vision(self, query):
159         msg = SetContent.Request()
160         msg.api_key = api_key
161         msg.base_url = base_url
162         msg.model = vllm_model
163         msg.prompt = VLLM_PROMPT
164         msg.query = query
165         self.get_logger().info('vision: %s' % query)
166         res = self.send_request(self.set_vllm_content_client, msg)
167         return res.message
```

Processes visual queries using the Vision Language Model (VLLM).

- ① Parameter Reception: Accepts a query string, such as “What do you see?”
- ② Service Call: Sends a request to the VLLM service via `set_vllm_content_client`.
- ③ Result Return: Returns the response message from the Vision Language Model.

◆ `play_audio_finish_callback(self, msg)` Method:

```
228     def play_audio_finish_callback(self, msg):
229         # msg = SetBool.Request()
230         # msg.data = True
231         # self.send_request(self.awake_client, msg)
232         # msg = SetInt32.Request()
233         # msg.data = 1
234         # self.send_request(self.set_mode_client, msg)
235         self.play_audio_finish = msg.data
```

Handles the callback notification for audio playback completion

◆ `process` Method

```

178 def process(self):
179     while self.running:
180         if self.llm_result:
181             self.interrupt = False
182             msg = String()
183             if 'action' in self.llm_result: # 如果有对应的行为返回那么就提取处理
184                 result = json.loads(self.llm_result[self.llm_result.find('{'):self.llm_result.find('}')+1])
185                 if 'response' in result:
186                     msg.data = result['response']
187                     self.tts_text_pub.publish(msg)
188                 if 'action' in result:
189                     action = result['action']
190                     self.get_logger().info(f'vllm action: {action}')
191                     for a in action:
192                         if 'move' in a:
193                             self.reach_goal = False
194                             eval(f'self.{a}')
195                             while not self.reach_goal:
196                                 if self.interrupt:
197                                     self.get_logger().info('interrupt')
198                                     break
199                                 # self.get_logger().info('waiting for reach goal')
200                                 time.sleep(1)
201                         elif 'vision' in a:
202                             res = eval(f'self.{a}')
203                             self.response_text = res
204                             self.get_logger().info(f'vllm response: {res}')
205                         elif 'play_audio' in a:
206                             eval(f'self.{a}')
207                         if self.interrupt:
208                             self.get_logger().info('interrupt')
209                             break
210                     else: # 没有对应的行为，只回答
211                         msg.data = self.llm_result
212                         self.tts_text_pub.publish(msg)
213                         self.action_finish = True
214                         self.llm_result = ''
215             else:
216                 time.sleep(0.01)

```

① Check LLM result; if self.llm_result: Check whether self.llm_result contains any content. If it is not empty, it indicates there is a new LLM result to process; otherwise, enter a short sleep to wait for new results.

② Check whether the result contains an action command;; if 'action' in self.llm_result: If the returned result contains the "action" field, it indicates that, in addition to the reply text, there are action commands that need to be executed.

③ Extract and Parse JSON Data: Locate the start and end positions of the JSON segment within the string using string search methods, then parse it into a dictionary with json.loads. Extract data such as "response" and "action" from the dictionary.

④ Handle Text Replies and Action Execution:

If "response" is present in the parsed result, assign its content to a message and publish it on the tts_text_pub topic to trigger TTS playback.

⑤ Movement Commands

If the action string contains "move", first set `self.reach_goal` to False, then call the corresponding movement function (executed dynamically using `eval`).

Enter a loop to wait until `self.reach_goal` becomes True (indicating the robot has reached its target).

During this wait, monitor `self.interrupt` to support interruption.

⑥ Vision Detection

If "vision" is included, execute the corresponding vision function, assign its return value to `self.response_text`, and log the result.

⑦ Play the Audio

If it contains "play_audio", directly call the corresponding audio playback function.

⑧ Interrupt Check: Continuously monitor the `self.interrupt` flag throughout the execution of each action. If an interrupt is triggered during waiting or action execution, promptly stop the current action and exit the corresponding loop.

⑨ Mark Completion and Switch Mode: After processing the current LLM result—regardless of whether it includes an action—set `self.action_finish` to True and clear `self.llm_result`. After in the loop, once both `self.play_audio_finish` and `self.action_finish` are True, reset these flags and send a mode-switch request via `set_mode_client` to transition to the next state.

Finally, when `self.running` is set to False, exit the loop and shut down the ROS system.

◆ Main Method

```
225 def main():
226     node = VLLMNavigation('vllm_navigation')
227     executor = MultiThreadedExecutor()
228     executor.add_node(node)
229     executor.spin()
230     node.destroy_node()
```

- ① Create an instance of the VLLMObjectTransport ROS 2 node.
- ② Uses a multithreaded executor to handle node tasks.
- ③ Calls executor.spin() to start processing ROS events.
- ④ Destroys the node when finished by calling node.destroy_node().

7. FAQ

7.1 Launching RVIZ on the Virtual Machine

If the game is lagging or performing poorly, you can display RVIZ on the virtual machine as follows. Refer to the instructions in '**12. Mapping Lesson\ Lesson 4 slam_toolbox Mapping Algorithm\ 2.Mapping Operation Steps**' for placing the necessary files in the virtual machine under:

- 1) Enter the following command and press Enter to initiate smart home assistant game:

```
ros2 launch large_models vllm_navigation_virtual.launch.py
```

```

/bin/zsh
/bin/zsh 80x24
-----
当前环境是ROS2\|The current environment is ROS2
-----
LIDAR:    MS200
CAMERA:   ascamera
MACHINE:  MentorPi_Mecanum
HOST:     /
MASTER:   /
ROS_DOMAIN_ID: 0
ASR_LANGUAGE: Chinese
VERSION:  |V2.0.0|2025-05-12|
[ 09:38:20 ]
ros2 launch large_models vllm_navigation_virtual.launch.py

```

- 2) Click  in the top-left corner to open the terminal.
- 3) Input the command below to launch the rviz tool to display the navigation process:

ros2 launch navigation rviz_navigation.launch.py

```
ubuntu@ubuntu-virtual-machine:~$ ros2 launch navigation rviz_navigation.launch.py
```

